



Chapter 9

Project 3.1: Data Cleaning Base Application

Data validation, cleaning, converting, and standardizing are steps required to transform raw data acquired from source applications into something that can be used for analytical purposes. Since we started using a small data set of very clean data, we may need to improvise a bit to create some "dirty" raw data. A good alternative is to search for more complicated, raw data.

This chapter will guide you through the design of a data cleaning application, separate from the raw data acquisition. Many details of cleaning, converting, and standardizing will be left for subsequent projects. This initial project creates a foundation that will be extended by adding features. The idea is to prepare for the goal of a complete data pipeline that starts with acquisition and passes the data through a separate cleaning stage. We want to exploit the Linux principle of having applications connected by a shared buffer, often referred to as a shell pipeline.

This chapter will cover a number of skills related to the design of data validation and cleaning applications:

- CLI architecture and how to design a pipeline of processes
- The core concepts of validating, cleaning, converting, and standardizing raw data

We won't address all the aspects of converting and standardizing data in this chapter. Projects in [*Chapter 10, Data Cleaning Features*](#) will expand on many conversion topics. The project in [*Chapter 12, Project 3.8: Integrated Data Acquisition Web Service*](#) will address the integrated pipeline idea. For now, we want to build an adaptable base application that can be extended to add features.

We'll start with a description of an idealized data cleaning application.

9.1 Description

We need to build a data validating, cleaning, and standardizing application. A data inspection notebook is a handy starting point for this design work. The goal is a fully-automated application to reflect the lessons learned from inspecting the data.

A data preparation pipeline has the following conceptual tasks:

- Validate the acquired source text to be sure it's usable and to mark invalid data for remediation.
- Clean any invalid raw data where necessary; this expands the available data in those cases where sensible cleaning can be defined.
- Convert the validated and cleaned source data from text (or bytes) to usable Python objects.
- Where necessary, standardize the code or ranges of source data. The requirements here vary with the problem domain.

The goal is to create clean, standardized data for subsequent analysis. Surprises occur all the time. There are several sources:

- Technical problems with file formats of the upstream software. The intent of the acquisition program is to isolate physical format issues.
- Data representation problems with the source data. The intent of this project is to isolate the validity and standardization of the values.

Once cleaned, the data itself may still contain surprising relationships, trends, or distributions. This is discovered with later projects that create analytic notebooks and reports. Sometimes a surprise comes from finding the *Null Hypothesis* is true and the data only shows insignificant random variation.

In many practical cases, the first three steps — validate, clean, and convert — are often combined into a single function call. For example, when dealing with numeric values, the `int()` or `float()` functions will validate and convert a value, raising an exception for invalid numbers.

In a few edge cases, these steps need to be considered in isolation – often because there's a tangled interaction between validation and cleaning.

For example, some data is plagued by dropping the leading zeros from US postal codes. This can be a tangled problem where the data is superficially invalid but can be reliably cleaned before attempting validation. In this case, validating the postal code against an official list of codes comes after cleaning, not before. Since the data will remain as text, there's no actual conversion step after the clean-and-validate composite step.

9.1.1 User experience

The overall **User Experience (UX)** will be two command-line applications. The first application will acquire the raw data, and the second will clean the data. Each has options to fine-tune the `acquire` and `cleanse` steps.

There are several variations on the `acquire` command, shown in earlier chapters. Most notably, [*Chapter 3, Project 1.1: Data Acquisition Base Application*](#), [*Chapter 4, Data Acquisition Features: Web APIs and Scraping*](#), and [*Chapter 5, Data Acquisition Features: SQL Database*](#).

For the **clean** application, the expected command-line should like something like this:

```
% python src/clean.py -o analysis -i quartet/Series_1.ndjson
```

The `-o analysis` specifies a directory into which the resulting clean data is written.

The `-i quartet/Series_1.ndjson` specifies the path to the source data file. This is a file written by the acquisition application.

Note that we're not using a positional argument to name the input file. The use of a positional argument for a filename is a common provision in many — but not all — Linux commands. The reason to avoid positional arguments is to make it easier to adapt this to become part of a pipeline of processing stages.

Specifically, we'd like the following to work, also:

```
% python src/acquire.py -s Series_1Pair --csv source.csv | \
python src/clean.py -o analysis/Series_1.ndjson
```

This shell line has two commands, one to do the raw data acquisition, and the other to perform the validation and cleaning. The acquisition command uses the `-s Series_1Pair` argument to name a specific series extraction class. This class will be used to create a single series as output. The `--csv source.csv` argument names the input file to process. Other options could name RESTful APIs or provide a database connection string.

The second command reads the output from the first command and writes this to a file. The file is named by the `-o` argument value in the second command.

This pipeline concept, made available with the shell's `|` operator, means these two processes will run concurrently. This means data is passed from one process to the other as it becomes available. For very large source files, cleaning data as it's being acquired can reduce processing time dramatically.

In **[Project 3.6: Integration to create an acquisition pipeline](#)** we'll expand on this design to include some ideas for concurrent processing.

Now that we've seen an overview of the application's purpose, let's take a look at the source data.

9.1.2 Source data

The earlier projects produced source data in an approximately consistent format. These projects focused on acquiring data that is text. The individual samples were transformed into small JSON-friendly dictionaries, using the NDJSON format. This can simplify the validation and cleaning operation.

The NDJSON file format is described at <http://ndjson.org> and <https://jsonlines.org>.

There are two design principles behind the **acquire** application:

- Preserve the original source data as much as possible.
- Perform the fewest text transformations needed during acquisition.

Preserving the source data makes it slightly easier to locate problems when there are unexpected changes to source applications. Minimizing

the text transformations, similarly, keeps the data closer to the source. Moving from a variety of representations to a single representation simplifies the data cleaning and transformation steps.

All of the data acquisition projects involve some kind of textual transformation from a source representation to ND JSON.

Chapter	Section	Source
<u>3</u>	<u>Chapter 3, Project 1.1: Data Acquisition Base Application</u>	CSV parsing
<u>4</u>	<u>Project 1.2: Acquire data from a web service</u>	Zipped CSV or JSON
<u>4</u>	<u>Project 1.3: Scrape data from a web page</u>	HTML
<u>5</u>	<u>Project 1.5: Acquire data from a SQL extract</u>	SQL Extract

In some cases — i.e., extracting HTML — the textual changes to peel the markup away from the data is profound. The SQL database extract involves undoing the database’s internal representation of numbers or dates and writing the values as text. In some cases, the text transformations are minor.

9.1.3 Result data

The cleaned output files will be ND JSON; similar to the raw input files. We’ll address this output file format in detail in [Chapter 11, Project 3.7: Interim Data Persistence](#). For this project, it’s easiest to stick with writing the JSON representation of a Pydantic dataclass.

For Python's native `dataclasses`, the `dataclasses.asdict()` function will produce a dictionary from a dataclass instance. The `json.dumps()` function will convert this to text in JSON syntax.

For Pydantic dataclasses, however, the `asdict()` function can't be used. There's no built-in method for emitting the JSON representation of a `pydantic` dataclass instance.

For version 1 of **Pydantic**, a slight change is required to write ND JSON. An expression like the following will emit a JSON serialization of a `pydantic` dataclass:

```
import json
from pydantic.json import pydantic_encoder
from typing import TextIO, Iterable

import analysis_model

def persist_samples(
    target_file: TextIO,
    analysis_samples: Iterable[analysis_model.SeriesSample | None]
) -> int:
    count = 0
    for result in filter(None, analysis_samples):
        target_file.write(json.dumps(result, default=pydantic_encoder))
        target_file.write("\n")
        count += 1
    return count
```

This central feature is the `default=pydantic_encoder` argument value for the `json.dumps()` function. This will handle the proper decoding of the dataclass structure into JSON notation.

For version 2 of **pydantic**, there will be a slightly different approach. This makes use of a `RootModel[classname](object)` construct to extract the root model for a given class from an object. In this case, `RootModel[SeriesSample](result).model_dump()` will create a root model that can emit a nested dictionary structure. No special `pydantic_encoder` will be required for version 2.

Now that we've looked at the inputs and outputs, we can survey the processing concepts. Additional processing details will wait for later projects.

9.1.4 Conversions and processing

For this project, we're trying to minimize the processing complications. In the next chapter, [*Chapter 10, Data Cleaning Features*](#), we'll look at a number of additional processing requirements that will add complications. As a teaser for the projects in the next chapter, we'll describe some of the kinds of field-level validation, cleaning, and conversion that may be required.

One example we've focused on, Anscombe's Quartet data, needs to be converted to a series of floating-point values. While this is painfully obvious, we've held off on the conversion from the text to the Python `float` object to illustrate the more general principle of separating the complications of acquiring data from analyzing the data. The output from this application will have each resulting ND JSON document with `float` values instead of `string` values.

The distinction in the JSON documents will be tiny: the use of `"` for the raw-data strings. This will be omitted for the `float` values.

This tiny detail is important because every data set will have distinct conversion requirements. The data inspection notebook will reveal data domains like text, integers, date-time stamps, durations, and a mixture of more specialized domains. It's essential to examine the data before trusting any schema definition or documentation about the data.

We'll look at three common kinds of complications:

- Fields that must be decomposed.
- Fields which must be composed.
- Unions of sample types in a single collection.
- "Opaque" codes used to replace particularly sensitive information.

One complication is when multiple source values are collapsed into a single field. This single source value will need to be decomposed into multiple values for analysis. With the very clean data sets available from Kaggle, a need for decomposition is unusual. Enterprise data sets, on the other hand, will often have fields that are not properly decomposed into atomic values, reflecting optimizations or legacy processing requirements. For example, a product ID code may include a line of business and a year of introduction as part of the code. For example, a boat's hull ID

number might include "421880182," meaning it's a 42-foot hull, serial number 188, completed in January 1982. Three disparate items were all coded as digits. For analytical purposes, it may be necessary to separate the items that comprise the coded value. In other cases, several source fields will need to be combined. An example data set where a timestamp is decomposed into three separate fields can be found when looking at tidal data.

See https://tidesandcurrents.noaa.gov/tide_predictions.html for Tide Predictions around the US. This site supports downloads in a variety of formats, as well as RESTful API requests for tide predictions.

Each of the tidal events in an annual tide table has a timestamp. The timestamp is decomposed into three separate fields: the date, the day of the week, and the local time. The day of the week is helpful, but it is entirely derived from the date. The date and time need to be combined into a single datetime value to make this data useful. It's common to use `datetime.combine(date, time)` to merge separate date and time values into a single value.

Sometimes a data set will have records of a variety of subtypes merged into a single collection. The various types are often discriminated from each other by the values of a field. A financial application might include a mixture of invoices and payments; many fields overlap, but the meanings of these two transaction types are dramatically different. A single field with a code value of "I" or "P" may be the only way to distinguish the types of business records represented.

When multiple subtypes are present, the collection can be called a *discriminated union* of subtypes; sometimes simply called a **union**. The discriminator and the subtypes suggest a class hierarchy is required to describe the variety of sample types. A common base class is needed to describe the common fields, including the discriminator. Each subclass has a distinct definition for the fields unique to the subclass.

One additional complication stems from data sources with "opaque" data. These are string fields that can be used for equality comparison, but nothing else. These values are often the result of a data analysis approach called **masking**, **deidentification**, or **pseudonymization**. This is sometimes also called "tokenizing" because an opaque token has replaced the sensitive data. In banking, for example, it's common for analytical data to

have account numbers or payment card numbers transformed into opaque values. These can be used to aggregate behavior, but cannot be used to identify an individual account holder or payment card. These fields must be treated as strings, and no other processing can be done.

For now, we'll defer the implementation details of these complications to a later chapter. The ideas should inform design decisions for the initial, foundational application.

In addition to clean valid data, the application needs to produce information about the invalid data. Next, we'll look at the logs and error reports.

9.1.5 Error reports

The central feature of this application is to output files with valid, useful data for analytic purposes. We've left off some details of what happens when an acquired document isn't actually usable.

Here are a number of choices related to the observability of invalid data:

- Raise an overall exception and stop. This is appropriate when working with carefully-curated data sets like the Anscombe Quartet.
- Make all of the bad data observable, either through the log or by writing bad data to a separate rejected samples file.
- Silently reject the bad data. This is often used with large data sources where there is no curation or quality control over the source.

In all cases, the summary counts of acquired data, usable analytic data, and cleaned, and rejected data are essential. It's imperative to be sure the number of raw records read is accounted for, and the provenance of cleaned and rejected data is clear. The summary counts, in many cases, are the primary way to observe changes in data sources. A non-zero error count, may be so important that it's used as the final exit status code for the cleaning application.

In addition to the observability of bad data, we may be able to clean the source data. There are several choices here, also:

- Log the details of each object where cleaning is done. This is often used with data coming from a spreadsheet where the unexpected data may be rows that need to be corrected manually.

- Count the number of items cleaned without the supporting details.
This is often used with large data sources where changes are frequent.
- Quietly clean the bad data as an expected, normal operational step.
This is often used when raw data comes directly from measurement devices in unreliable environments, perhaps in space, or at the bottom of the ocean.

Further, each field may have distinct rules for whether or not cleaning bad data is a significant concern or a common, expected operation. The intersection of observability and automated cleaning has a large number of alternatives.

The solutions to data cleaning and standardization are often a matter of deep, ongoing conversations with users. Each data acquisition pipeline is unique with regard to error reporting and data cleaning.

It's sometimes necessary to have a command-line option to choose between logging each error or simply summarizing the number of errors. Additionally, the application might return a non-zero exit code when any bad records are found; this permits a parent application (e.g., a shell script) to stop processing in the presence of errors.

We've looked at the overall processing, the source files, the result files, and some of the error-reporting alternatives that might be used. In the next section, we'll look at some design approaches we can use to implement this application.

9.2 Approach

We'll take some guidance from the C4 model (<https://c4model.com>) when looking at our approach.

- **Context:** For this project, the context diagram has expanded to three use cases: acquire, inspect, and clean.
- **Containers:** There's one container for the various applications: the user's personal computer.
- **Components:** There are two significantly different collections of software components: the acquisition program and the cleaning program.
- **Code:** We'll touch on this to provide some suggested directions.

A context diagram for this application is shown in [Figure 9.1](#).

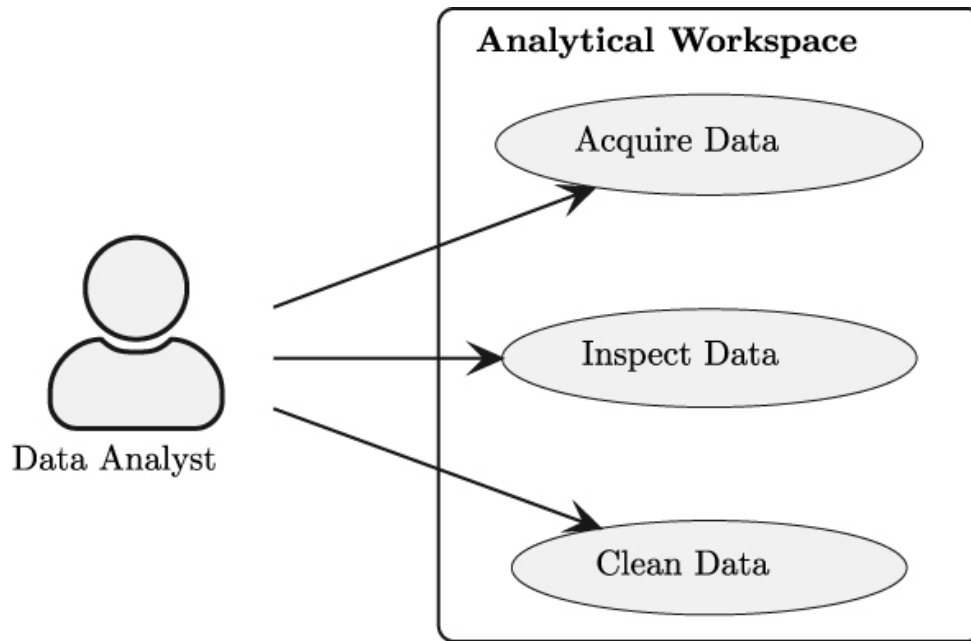


Figure 9.1: Context Diagram

A component diagram for the conversion application isn't going to be as complicated as the component diagrams for acquisition applications. One reason for this is there are no choices for reading, extracting, or downloading raw data files. The source files are the ND JSON files created by the acquisition application.

The second reason the conversion programs tend to be simpler is they often rely on built-in Python-type definitions, and packages like `pydantic` to provide the needed conversion processing. The complications of parsing HTML or XML sources were isolated in the acquisition layer, permitting this application to focus on the problem domain data types and relationships.

The components for this application are shown in **Figure 9.2**.

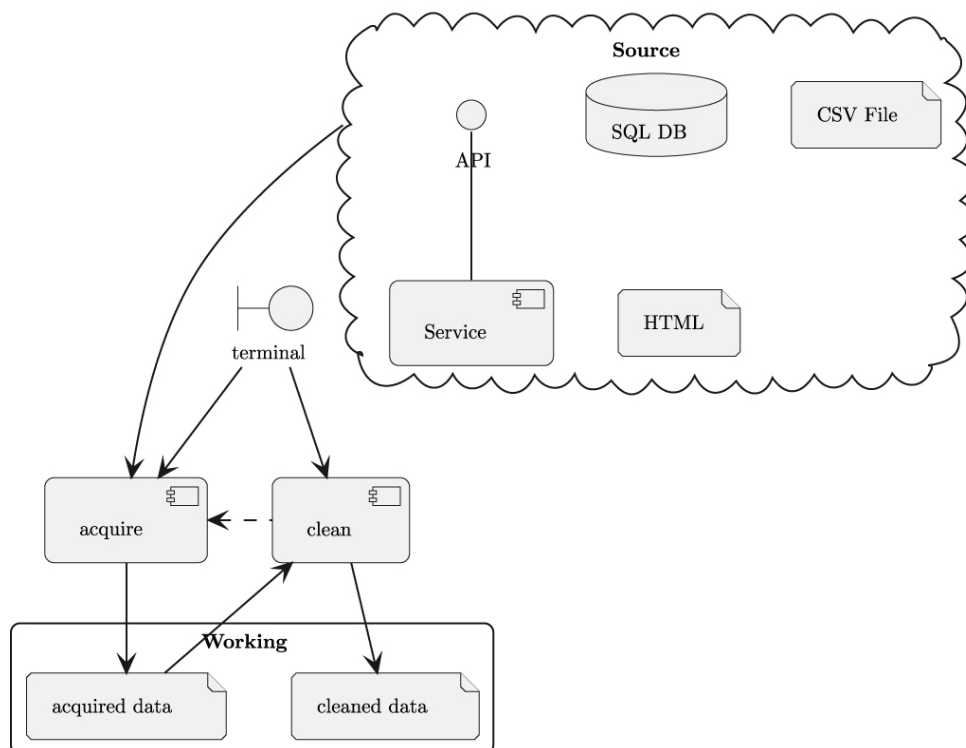


Figure 9.2: Component Diagram

Note that we've used a dotted "depends on" arrow. This does not show the data flow from **acquire** to **clean**. It shows how the **clean** application depends on the **acquire** application's output.

The design for the **clean** application often involves an almost purely functional design. Class definitions — of course — can be used. Classes don't seem to be helpful when the application processing involves stateless, immutable objects.

In rare cases, a cleaning application will be required to perform dramatic reorganizations of data. It may be necessary to accumulate details from a variety of transactions, updating the state of a composite object. For example, there may be multiple payments for an invoice that must be combined for reconciliation purposes. In this kind of application, associating payments and invoices may require working through sophisticated matching rules.

Note that the **clean** application and the **acquire** application will both share a common set of dataclasses. These classes represent the source data, the output from the **acquire** application. They also define the input to the **clean** application. A separate set of dataclasses represent the working values used for later analysis applications.

Our goal is to create three modules:

- `clean.py` : The main application.
- `analytical_model.py` : A module with dataclass definitions for the pure-Python objects that we'll be working with. These classes will — generally — be created from JSON-friendly dictionaries with string values.
- `conversions.py` : A module with any specialized validation, cleaning, and conversion functions.

If needed, any application-specific conversion functions may be required to transform source values to "clean," usable Python objects. If this can't be done, the function can instead raise `ValueError` exceptions for invalid data, following the established pattern for functions like Python's built-in `float()` function. Additionally, `TypeError` exceptions may be helpful when the object — as a whole — is invalid. In some cases, the `assert` statement is used, and an `AssertionError` may be raised to indicate invalid data.

For this baseline application, we'll stick to the simpler and more common design pattern. We'll look at individual functions that combine validation and cleaning.

9.2.1 Model module refactoring

We appear to have two distinct models: the "as-acquired" model with text fields, and the "to-be-analyzed" model with proper Python types, like `float` and `int`. The presence of multiple variations on the model means we either need a lot of distinct class names, or two distinct modules as namespaces to keep the classes organized.

The cleaning application is the only application where the acquire and analysis models are **both** used. All other applications either acquire raw data or work with clean analysis data.

The previous examples had a single `model.py` module with dataclasses for the acquired data. At this point, it has become more clear that this was not a great long-term decision. Because there are two distinct variations on the data model, the generic `model` module name needs to be refactored. To distinguish the acquired data model from the analytic data

model, a prefix should be adequate: the module names can be `acquire_model` and `analysis_model`.

(The English parts of speech don't match exactly. We'd rather not have to type "acquisition_model". The slightly shorter name seems easier to work with and clear enough.)

Within these two model files, the class names can be the same. We might have names `acquire_model.SeriesSample` and `analysis_model.SeriesSample` as distinct classes.

To an extent, we can sometimes copy the acquired model module to create the analysis model module. We'd need to change `from dataclasses import dataclass` to the **Pydantic** version, `from pydantic import dataclass`. This is a very small change, which makes it easy to start with. In some older versions of **Pydantic** and **mypy**, the **Pydantic** version of `dataclass` doesn't expose the attribute types in a way that is transparent to the **mypy** tool.

In many cases, it can work out well to import `BaseModel` and use this as the parent class for the analytic models. Using the `pydantic.BaseModel` parent class often has a better coexistence with the **mypy** tool. This requires a larger change when upgrading from dataclasses to leverage the **pydantic** package. Since it's beneficial when using the **mypy** tool, it's the path we recommend following.

This **Pydantic** version of `dataclass` introduces a separate validator method that will be used (automatically) to process fields. For simple class definitions with a relatively clear mapping from the acquire class to the analysis class, a small change is required to the class definition.

One common design pattern for this new analysis model class is shown in the following example for **Pydantic** version 1:

```
from pydantic import validator, BaseModel, Field

class SeriesSample(BaseModel):
    """
    An individual sample value.
    """
    x: float = Field(title="The x attribute", ge=0.0)
    y: float = Field(title="The y attribute", ge=0.0)
```

```

@validator('x', 'y', pre=True)
def clean_value(cls, value: str | float) -> str:
    match value:
        case str():
            for char in "\N{ZERO WIDTH SPACE}":
                value = value.replace(char, "")
            return value
        case float():
            return value

```

This design defines a class-level method, `clean_value()`, to handle cleaning the source data when it's a string. The validator has the `@validator()` decorator to provide the attribute names to which this function applies, as well as the specific stage in the sequence of operations. In this case, `pre=True` means this validation applies **before** the individual fields are validated and converted to useful types.

This will be replaced by a number of much more flexible alternatives in **Pydantic** version 2. The newer release will step away from the `pre=True` syntax used to assure this is done prior to the built-in handler accessing the field.

The Pydantic 2 release will introduce a radically new approach using annotations to specify validation rules. It will also retain a decorator that's very similar to the old version 1 validation.

One migration path is to replace `validator` with `field_validator`. This will require changing the `pre=True` or `post=True` with a more universal `mode='before'` or `mode='after'`. This new approach permits writing field validators that “wrap” the conversion handler with both before and after processing.

To use **Pydantic** version two, use

`@field_validator('x', 'y', mode='before')` to replace the `@validator` decorator in the example. The `import` must also change to reflect the new name of the decorator.

This validator function handles the case where the string version of source data can include Unicode `U+200B`, a special character called the zero-width space. In Python, we can use `"\N{ZERO WIDTH SPACE}"` to make this character visible. While lengthy, this name seems better than the obscure `"\u200b"`.

(See <https://www.fileformat.info/info/unicode/char/200b/index.htm> for details of this character.)

When a function works in the `pre=True` or `mode='before'` phase, then **pydantic** will automatically apply the final conversion function to complete the essential work of validation and conversion. This additional validator function can be designed, then, to focus narrowly only on cleaning the raw data.

The idea of a validator function must reflect two separate use cases for this class:

1. Cleaning and converting acquired data, generally strings, to more useful analytical data types.
2. Loading already cleaned analytical data, where type conversion is not required.

Our primary interest at this time is in the first use case, cleaning and conversion. Later, starting in chapter ***Chapter 13, Project 4.1: Visual Analysis Techniques*** we'll switch over to the second case, loading clean data.

These two use cases are reflected in the type hint for the validator function. The parameter is defined as `value: str | float`. The first use case, conversion, expects a value of type `str`. The second use case, loading cleaned data, expects a cleaned value of type `float`. This kind of type of union is helpful with validator functions.

Instances of the analytic model will be built from `acquire_model` objects. Because the acquired model uses `dataclasses`, we can leverage the `dataclasses.asdict()` function to transform a source object into a dictionary. This can be used to perform Pydantic validation and conversion to create the analytic model objects.

We can add the following method in the dataclass definition:

```
@classmethod
def from_acquire_dataclass(
    cls,
    acquired: acquire_model.SeriesSample
```



```

    ) -> "SeriesSample":
        return SeriesSample(**asdict(acquired))

```

This method extracts a dictionary from the acquired data model's version of the `SeriesSample` class and uses it to create an instance of the analytic model's variation of this class. This method pushes all of the validation and conversion work to the **Pydantic** declarations. This method also requires `from dataclasses import asdict` to introduce the needed `asdict()` function.

In cases where the field names don't match, or some other transformation is required, a more complicated dictionary builder can replace the `asdict(acquired)` processing. We'll see examples of this in [Chapter 10, Data Cleaning Features](#), where acquired fields need to be combined before they can be converted.

We'll revisit some aspects of this design decision in [Chapter 11, Project 3.7: Interim Data Persistence](#). First, however, we'll look at **pydantic** version 2 validation, which offers a somewhat more explicit path to validation functions.

9.2.2 Pydantic V2 validation

While **pydantic** version 2 will offer a `@field_validator` decorator that's very similar to the legacy `@validator` decorator, this approach suffers from an irksome problem. It can be confusing to have the decorator listing the fields to which the validation rule applies. Some confusion can arise because of the separation between the field definition and the function that validates the values for the field. In our example class, the validator applies to the `x` and `y` fields, a detail that might be difficult to spot when first looking at the class.

The newer design pattern for the analysis model class is shown in the following example for Pydantic version 2:

```

from pydantic import BaseModel
from pydantic.functional_validators import field_validator, BeforeValidator

from typing import Annotated

def clean_value(value: str | float) -> str | float:
    match value:

```

```

    case str():
        for char in "\N{ZERO WIDTH SPACE}":
            value = value.replace(char, "")
        return value
    case float():
        return value

class SeriesSample(BaseModel):
    x: Annotated[float, BeforeValidator(clean_value)]
    y: Annotated[float, BeforeValidator(clean_value)]

```

We've omitted the `from_acquire_dataclass()` method definition, since it doesn't change.

The cleaning function is defined outside the class, making it more easily reused in a complicated application where a number of rules may be widely reused in several models. The `Annotated[]` type hint combines the base type with a sequence of validator objects. In this example, the base type is `float` and the validator objects are `BeforeValidator` objects that contain the function to apply.

To reduce the obvious duplication, a `TypeAlias` can be used. For example,

```

from typing import Annotated, TypeAlias

CleanFloat: TypeAlias = Annotated[float, BeforeValidator(clean_value)]

```

Using an alias permits the model to use the type hint `CleanFloat`. For example `x: CleanFloat`.

Further, the `Annotated` hints are composable. An annotation can add features to a previously-defined annotation. This ability to build more sophisticated annotations on top of foundational annotations offers a great deal of promise for defining classes in a succinct and expressive fashion.

Now that we've seen how to implement a single validation, we need to consider the alternatives, and how many different kinds of validation functions an application might need.

9.2.3 Validation function design

The `pydantic` package offers a vast number of built-in conversions based entirely on annotations. While these can cover a large number of common cases, there are still some situations that require special validators, and perhaps even special type definitions.

In ***Conversions and processing***, we considered some of the kinds of processing that might be required. These included the following kinds of conversions:

- Decomposing source fields into their atomic components.
- Merging separated source fields to create proper value. This is common with dates and times, for example.
- Multiple subentities may be present in a feed of samples. This can be called a discriminated union: the feed as a whole is a unique of disjoint types, and a discriminator value (or values) distinguishes the various subtypes.
- A field may be a “token” used to deidentify something about the original source. For example, a replacement token for a driver’s license number may replace the real government-issued number to make the individual anonymous.

Additionally, we may have observability considerations that lead us to write our own a unique validator that can write needed log entries or update counters showing how many times a particular validation found problems. This enhanced visibility can help pinpoint problems with data that is often irregular or suffers from poor quality control.

We’ll dive into these concepts more deeply in ***Chapter 10, Data Cleaning Features***. In ***Chapter 10, Data Cleaning Features***, we’ll also look at features for handling primary and foreign keys. For now, we’ll focus on the built-in type conversion functions that are part of Python’s built-in functions, and the standard library. But we need to recognize that there are going to be extensions and exceptions.

We’ll look at the overall design approach in the next section.

9.2.4 Incremental design

The design of the cleaning application is difficult to finalize without detailed knowledge of the source data. This means the cleaning application depends on lessons learned by making a data inspection notebook. One

idealized workflow begins with “understand the requirements” and proceeds to “write the code,” treating these two activities as separate, isolated steps. This conceptual workflow is a bit of a fallacy. It’s often difficult to understand the requirements without a detailed examination of the actual source data to reveal the quirks and oddities that are present. The examination of the data often leads to the first drafts of data validation functions. In this case, the requirements will take the form of draft versions of the code, not a carefully-crafted document.

This leads to a kind of back-and-forth between *ad-hoc* inspection and a formal data cleaning application. This iterative work often leads to a module of functions to handle the problem domain’s data. This module can be shared by inspection notebooks as well as automated applications. Proper engineering follows the **DRY (Don’t Repeat Yourself)** principle: code should not be copied and pasted between modules. It should be put into a shared module so it can be reused properly.

In some cases, two data cleaning functions will be similar. Finding this suggests some kind of decomposition is appropriate to separate the common parts from the unique parts. The redesign and refactoring are made easier by having a suite of unit tests to confirm that no old functionality was broken when the functions were transformed to remove duplicated code.

The work of creating cleaning applications is iterative and incremental. Rare special cases are — well — rare, and won’t show up until well after the processing pipeline seems finished. The unexpected arrival special case data is something like birders seeing a bird outside its expected habitat. It helps to think of a data inspection notebook like a bird watcher’s immense spotting scope, used to look closely at one unexpected, rare bird, often in a flock of birds with similar feeding and roosting preferences. The presence of the rare bird becomes a new datapoint for ornithologists (and amateur enthusiasts). In the case of unexpected data, the inspection notebook’s lessons become a new code for the conversions module.

The overall main module in the data cleaning application will implement the **command-line interface (CLI)**. We’ll look at this in the next section.

9.2.5 CLI application

The UX for this application suggests that it operates in the following distinct contexts:

- As a standalone application. The user runs the `src/acquire.py` program. Then, the user runs the `src/clean.py` program.
- As a stage in a processing pipeline. The user runs a shell command that pipes the output from the `src/acquire.py` program into the `src/clean.py` program. This is the subject of [***Project 3.6: Integration to create an acquisition pipeline.***](#)

This leads to the following two runtime contexts:

- When the application is provided an input path, it's being used as a stand-alone application.
- When no input path is provided, the application reads from `sys.stdin`.

A similar analysis can apply to the **acquire** application. If an output path is provided, the application creates and writes the named file. If no output path is provided, the application writes to `sys.stdout`.

One essential consequence of this is all logging **must** be written to `sys.stderr`.

Use **stdin** and **stdout** exclusively for application data, nothing else.

Use a consistent, easy-to-parse text format like ND JSON for application data.

Use **stderr** as the destination for all control and error messages.

This means `print()` may require the `file=sys.stderr` to direct debugging output to **stderr**. Or, avoid simple `print()` and use `logger.debug()` instead.

For this project, the stand-alone option is all that's needed. However, it's important to understand the alternatives that will be added in later projects. See [***Project 3.6: Integration to create an acquisition pipeline***](#) for this more tightly-integrated alternative.

Redirecting stdout

Python provides a handy tool for managing the choice between "write to an open file" and "write to **stdout**". It involves the following essential design principle.

Always provide file-like objects to functions and methods processing data.

This suggests a data-cleaning function like the following:

```
from typing import TextIO

def clean_all(acquire_file: TextIO, analysis_file: TextIO) -> None:
    ...
```

This function can use `json.loads()` to parse each document from the `acquire_file`. It uses `json.dumps()` to save each document to the `analysis_file` to be used for later analytics.

The overall application can then make a choice among four possible ways to use this `clean_all()` function:

- **Stand-alone:** This means `with` statements manage the open files created from the `Path` names provided as argument values.
- **Head of a pipeline:** A `with` statement can manage an open file passed to `acquire_file`. The value of `analysis_file` is `sys.stdout`.
- **Tail of a pipeline:** The acquired input file is `sys.stdin`. A `with` statement manages an open file (in write mode) for the `analysis_file`.
- **Middle of a pipeline:** The `acquire_file` is `sys.stdin`; the `analysis_file` is `sys.stdout`.

Now that we've looked at a number of technical approaches, we'll turn to the list of deliverables for this project in the next section.

9.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Application to clean some acquired data and apply simple conversions to a few fields. Later projects will add more complex validation rules.

We'll look at a few of these deliverables in a little more detail.

When starting a new kind of application, it often makes sense to start with acceptance tests. Later, when adding features, the new acceptance tests may be less important than new unit tests for the features. We'll start by looking at a new scenario for this new application.

9.3.1 Acceptance tests

As we noted in [*Chapter 4, Data Acquisition Features: Web APIs and Scraping*](#), we can provide a large block of text as part of a Gherkin scenario. This can be the contents of an input file. We can consider something like the following scenario.

```
Scenario: Valid file is recognized.  
    Given a file "example_1.ndjson" with the following content  
        """  
        {"x": "1.2", "y": "3.4"}  
        {"x": "five", "z": null}  
        """  
    When the clean tool is run  
    Then the output shows 1 good records  
    And the output shows 1 faulty records
```

This kind of scenario lets us define source documents with valid data. We can also define source documents with invalid data.

We can use the **Then** steps to confirm additional details of the output. For example, if we've decided to make all of the cleaning operations visible, the test scenario can confirm the output contains all of the cleanup operations that were applied.

The variety of bad data examples and the number of combinations of good and bad data suggest there can be a lot of scenarios for this kind of application. Each time new data shows up that is acquired, but cannot be cleaned, new examples will be added to these acceptance test cases.

It can, in some cases, be very helpful to publish the scenarios widely so all of the stakeholders can understand the data cleaning operations. The Gherkin language is designed to make it possible for people with limited technical skills to contribute to the test cases.

We also need scenarios to run the application from the command-line. The `When` step definition for these scenarios will be `subprocess.run()` to invoke the **clean** application, or to invoke a shell command that includes the **clean** application.

9.3.2 Unit tests for the model features

It's important to have automated unit tests for the model definition classes.

It's also important to **not** test the `pydantic` components. We don't, for example, need to test the ordinary string-to-float conversions the `pydantic` module already does; we can trust this works perfectly.

We **must** test the validator functions we've written. This means providing test cases to exercise the various features of the validators. Additionally, any overall `from_acquire_dataclass()` method needs to have test cases.

Each of these test scenarios works with a given acquired document with the raw data. When the `from_acquire_dataclass()` method is evaluated, then there may be an exception or a resulting analytic model document is created.

The exception testing can make use of the `pytest.raises()` context manager. The test is written using a `with` statement to capture the exception.

See <https://docs.pytest.org/en/7.2.x/how-to/assert.html#assertions-about-expected-exceptions> for examples.

Of course, we also need to test the processing that's being done. By design, there isn't very much processing involved in this kind of application. The bulk of the processing can be only a few lines of code to consume the raw model objects and produce the analytical objects. Most of the work will be delegated to modules like `json` and `pydantic`.

9.3.3 Application to clean data and create an NDJSON interim file

Now that we have acceptance and unit test suites, we'll need to create the `clean` application. Initially, we can create a place-holder application, just

to see the test suite fail. Then we can fill in the various pieces until the application – as a whole – works.

Flexibility is paramount in this application. In the next chapter, [*Chapter 10, Data Cleaning Features*](#), we will introduce a large number of data validation scenarios. In [*Chapter 11, Project 3.7: Interim Data Persistence*](#) we'll revisit the idea of saving the cleaned data. For now, it's imperative to create clean data; later, we can consider what format might be best.

9.4 Summary

This chapter has covered a number of aspects of data validation and cleaning applications:

- CLI architecture and how to design a simple pipeline of processes.
- The core concepts of validating, cleaning, converting, and standardizing raw data.

In the next chapter, we'll dive more deeply into a number of data cleaning and standardizing features. Those projects will all build on this base application framework. After those projects, the next two chapters will look a little more closely at the analytical data persistence choices, and provide an integrated web service for providing cleaned data to other stakeholders.

9.5 Extras

Here are some ideas for you to add to this project.

9.5.1 Create an output file with rejected samples

In [*Error reports*](#) we suggested there are times when it's appropriate to create a file of rejected samples. For the examples in this book — many of which are drawn from well-curated, carefully managed data sets — it can feel a bit odd to design an application that will reject data.

For enterprise applications, data rejection is a common need.

It can help to look at a data set like this: <https://datahub.io/core/co2-ppm>. This contains data same with measurements of CO2 levels measures

with units of ppm, parts per million.

This has some samples with an invalid number of days in the month. It has some samples where a monthly CO2 level wasn't recorded.

It can be insightful to use a rejection file to divide this data set into clearly usable records, and records that are not as clearly usable.

The output will **not** reflect the analysis model. These objects will reflect the acquire model; they are the items that would not convert properly from the acquired structure to the desired analysis structure.